

Compiling Optimal Retry Policies for Dependent Transformations

ANONYMOUS AUTHOR(S)

A concurrent transformation can prepare optimistically and retry after validation failure, or compute while excluding conflicting writes. Dependencies also make the next ready transformation a control choice. We study persistent completion, fixed opaque work and declared guarded-work premiums. A potential proves that serial retry policies attain the minimum worst work of a preparation/guard interface with retained preparations and batch validation. For serial atomic callbacks, a matching adversary uses at most B actual external writes, including shared internal monitors. Specializing a known reflected recurrence, we compile all-budget values and policies. A basis with input costs as slopes gives a tight $2n$ run bound and $2^n \text{poly}(\lambda)$ bit-time construction/checking for binary input length λ . Threshold synthesis is NP-complete even for $B = 1$, two layers and unit premiums; unique-order and cost-compatible DAGs permit $O(|E| + n \log(n + 1))$ compilation and generated-certificate checking. Fixed evaluation checks 4,232 complete-primitive inputs and 19,644 native Java paths on 48 generated DAGs plus controls. Among 192 priced roots, adaptation beats the best fixed topological order on two, with maximum total-work reduction 2.08%. Specified-budget alternatives expose when constructing all budgets is unnecessary. Practical prevalence and elapsed-time benefit remain unmeasured.

CCS Concepts: • **Software and its engineering** → **Formal software verification**; *Concurrent programming structures*.

Additional Key Words and Phrases: program synthesis, concurrency, retries, precedence constraints, quantitative games

1 Introduction

A transformation can prepare optimistically and conditionally commit, or compute under protection. From a DAG of persistent predecessor outputs and declared prices, we compile ready-job/mode choices from the remaining failure allowance. Figure 1 shows a minimum-size separation; fixed-evaluation gains are smaller.



Fig. 1. Four jobs, two possible failures. Nodes show job IDs and (c_i, p_i) . After job 0 succeeds, the remaining budget changes the optimal next job and mode; at zero budget all remaining jobs finish fast. Total work is 24 adaptively versus 26 for the best fixed order with optimal adaptive modes.

A potential makes serial policies optimal despite retained guards/preparations and batch validation. A matching serial-callback adversary uses at most B actual writes, handling colliding bins and repeated stale rejection.

A reflected recurrence [21] specializes to all-budget compilation with tight $2n$ run bounds and single-exponential bit complexity. Even $B = 1$ synthesis is NP-complete. Prescribed orders admit persistent compilation; unique-order and cost-compatible DAGs admit $O(|E| + n \log(n + 1))$ compilation and generated-certificate checking. Adaptation first helps at four jobs and two failures.

Adaptation, causal timing and parametric concavity are established [2, 12, 24, 25]. Our abstraction and compilation require opaque fixed work, guaranteed protection and declared prices. The map supplies no write bound; application prevalence and elapsed-time gains remain unmeasured.

2 The Preparation and Guard Interface

Jobs $J = \{1, \dots, n\}$ form a DAG with incoming neighbors $\text{Pred}(i)$. Examples and artifacts use zero-based identifiers. Each requires one foreground transformation. The completed set D is an ideal: a job is physically ready only when $\text{Pred}(i) \subseteq D$. Completion persists for the batch objective, although later external writes may change a committed live value. The objective is one linearized transformation per job, not a simultaneous all-object snapshot or commit.

A ready job invokes a pure opaque whole kernel of fixed work $w_i > 0$ on its input identity and captured immutable predecessor outputs, including their processing in w_i . Partial evaluation, premature preparation, incremental substitutes and external milestone discharge are excluded. A result is a nonforgeable capability tied to the job, identity and predecessor tuple. Commit stores the exact successful prepared/callback output before publishing completion; later live-object reads cannot reconstruct it. External writes change own-input identities without mutating captured outputs.

Kernel evaluation while protection excludes conflicting updates costs $w_i + p_i$, with $p_i \geq 0$; unprotected evaluation costs w_i . Incidental protection from a shared guard must use the same pricing rule. Acquisition is guaranteed to finish. Acquisition, release, inspection, validation, commit, copying and discard cost zero. The metric excludes their machine instructions, allocation, waiting and elapsed time. Rational prices can be uniformly scaled to integers.

An interface state is (x, b) with remaining invalidation allowance b . Initially $x_i = U$ and $b = B$. The five local states are

U	Unprepared, unguarded
C	Cached input/result pair, unguarded
L	Guarded, needs computation
LC	Guarded with a valid prepared result
D	Required foreground transformation committed

Nature resolves the program's history-dependent choices. Normal validation is always legal; invalidations consume tokens as below. Validation resolves a cached C result's usability; LC stays valid while guarded. Kernels require physical readiness, so cached jobs' predecessors remain completed. Unprepared jobs' guards may be acquired/released before readiness.

Individual operations. Preparation changes $U \rightarrow C$ for w_i , or $L \rightarrow LC$ for $w_i + p_i$. Free deterministic operations are acquisition $U \rightarrow L$, release $L \rightarrow U$ or $LC \rightarrow C$, commit $LC \rightarrow D$, and discard $C \rightarrow U$ or $LC \rightarrow L$. A conditional commit from C succeeds to D for zero cost, or, if $b > 0$, fails to U and spends one token. An additional retained-preparation failure to C may be offered. Acquisition/validation from C normally reaches LC, or spends one token and reaches L, retaining its guard for further decisions.

An atomic fresh callback completes $U \rightarrow D$ for $w_i + p_i$. An atomic cached callback from C completes for zero cost on a match, or spends one token and recomputes for $w_i + p_i$ on a mismatch. These operations affect only the selected job. Free cached inspection can report a match and remain C, or spend one token and become U. Conditional acquisition of an uncomputed snapshot can normally reach L, or spend one token and remain U; both outcomes cost zero.

Fixed batches. The program may select any subset of unguarded jobs for one atomic acquisition/validation. Selected U jobs become L. If K is the selected cached subset, Nature chooses $F \subseteq K$

with $|F| \leq b$: members of F become L, the others become LC, and b decreases by $|F|$. This grants all groups, even if a concrete interface offers fewer. Batch release and commit have the same accumulated cost as individual operations and add no all-object atomicity guarantee.

Interleaving, multiple preparations and held guards are allowed. Invocations must finish; noncompletion, including free cycles, costs infinity. The complete outcomes define the adversary contract. Restricted/correlated outcomes can reduce an application's optimum: synthesis retains an upper bound, while equality needs a matching environment. Section 5 supplies one for a callback subclass.

3 The Reduced Retry Game

Jobs $J = \{1, \dots, n\}$ have integer costs $a_i, c_i > 0, p_i \geq 0$: normal work, failed-attempt work and protection premium. The initial budget is an integer $B \geq 0$. A decision state (S, b) contains unfinished jobs and remaining allowance, initially (J, B) . Only $i \in \mathcal{A}(S) = \{i \in S : \text{Pred}(i) \cap S = \emptyset\}$ is available. Completion persists, so reachable unfinished sets are upsets. For proofs, extend the game to every subset, treating absent predecessors as completed.

The controller selects an available job and mode. *Protected* pays $a_i + p_i$ and reaches $(S \setminus \{i\}, b)$. *Fast* lets Nature choose normal completion for a_i , or, if $b > 0$, failure for c_i to $(S, b - 1)$. Normal completion is always legal. Both players observe outcomes/history. A failure spends global allowance, permits reselection and need not undo external writes.

Termination and residual costs. Decreasing rank $|S| + b$ gives termination within $n + B$ attempts. Subtracting the once-paid $A(S) = \sum_{i \in S} a_i$ leaves costs zero/ p_i/c_i for normal/protected/failed execution. Optimal worst excess satisfies

$$V(\emptyset, b) = V(S, 0) = 0, \quad (1)$$

$$V(S, b) = \min_{i \in \mathcal{A}(S)} \min \{p_i + V(S - i, b), \max[V(S - i, b), c_i + V(S, b - 1)]\}, \quad b > 0. \quad (2)$$

Here $S - i = S \setminus \{i\}$. With $N \leq 2^n$ reachable completion ideals, the recurrence has up to $(B + 1)N$ residual states and n candidate jobs each. Theorem 3 gives a separate complexity lower bound.

A minimizing deterministic action attains each finite perfect-information worst-path value. This does not address expected costs against an oblivious adversary.

Set $a_i = c_i = w_i$ for the primitive interface; the order/compiler results also allow $a_i \neq c_i$. The rank argument concerns macros; primitive programs may have free cycles.

LEMMA 1 (SUBSET MONOTONICITY). *For $T \subseteq S$ and any b , $V(T, b) \leq V(S, b)$.*

PROOF. Simulate a policy on S , completing omitted jobs normally or protected with nonnegative premium. Retained jobs relay actual actions, outcomes and token decrements. Physical and virtual unfinished sets satisfy $R = W \cap T$, making each virtually available retained job physically available. Each omitted job disappears in one action; after physical completion, finish the virtual remainder normally. Physical cost never exceeds virtual cost. Minimize over policies. $\square$

LEMMA 2 (CHEAPEST AVAILABLE FAST ACTION). *Fix a minimum- c_i job $i \in \mathcal{A}(S)$ and $b > 0$. For $F_j = \max\{V(S - j, b), c_j + V(S, b - 1)\}$, we have $F_i \leq F_j$ for every $j \in \mathcal{A}(S)$.*

PROOF. For $j \neq i$, j remains available after removing i , so choosing fast j in $S - i$ gives

$$V(S - i, b) \leq \max\{V(S - \{i, j\}, b), c_j + V(S - i, b - 1)\} \leq F_j,$$

where the final inequality follows from Lemma 1. The other branch satisfies $c_i + V(S, b - 1) \leq c_j + V(S, b - 1) \leq F_j$. Their maximum is F_i . The case $j = i$ is immediate. $\square$

Fast-action dominance neither chooses the protected job nor fixes completion order; the introduction's example requires adaptation.

3.1 Single-Failure Complexity

At $B = 1$, inspect a policy's no-failure path: a topological order π and its protected jobs H . After the only possible failure, every remaining job can finish fast with zero further excess, in the same remaining order. Thus its optimal continuation has value

$$C(\pi, H) = \max \left\{ \sum_{j \in H} p_j, \max_{i \notin H} \left(c_i + \sum_{\substack{j \in H \\ j \prec_{\pi} i}} p_j \right) \right\}, \quad (3)$$

omitting the second term when all jobs are protected. Minimizing over π, H gives $V(J, 1)$; hence an initially fixed order attains it. Yet finding that order is hard.

THEOREM 3 (RESTRICTED THRESHOLD COMPLEXITY). *At $B = 1$, deciding $V(J, 1) \leq K$ is NP-complete even with at most two DAG layers, maximum indegree two, all $p_i = 1$, and at most two positive instance-dependent failure-cost values. The reduction's numbers are polynomially bounded in its job count.*

PROOF. An order and mode bits form a polynomial certificate checked by Equation (3), proving membership. Reduce from CLIQUE [27]. Handle trivial instances first; otherwise $k \geq 2, m \geq \ell = \binom{k}{2}$. Add an isolated vertex, preserving clique existence and giving $n > k$ vertices. Set $K = n + m - \ell > k$. Each vertex becomes a job with $(c, p) = (K + 1, 1)$. Each edge becomes a job with $(c, p) = (K - k, 1)$ dependent on its two endpoints.

For a k -clique, protect its k vertices, then run its ℓ edges fast. A failure costs $k + (K - k) = K$; subsequently all jobs finish fast. With no failure, protect every remaining job, giving total premium $n + m - \ell = K$.

Conversely, on a policy's no-failure path of value at most K , every vertex must be protected: its fast failure alone would cost $K + 1$. Each fast edge must follow at most k protected jobs, since its failure costs $K - k$. Its endpoints therefore lie among the same first k protected vertices; interspersed protected edges cannot enlarge this set. If f edges are fast, no-failure premium $n + m - f \leq K$ implies $f \geq \ell$. At most k vertices contain at most ℓ distinct edges, so they form a clique. The DAG has $n + m$ jobs, $2m$ dependencies and costs at most $n + m + 1$. The full-work threshold is $A(J) + K$. $\square$

Hardness thus precedes the need for order adaptation and large budgets. It does not prove that ideal enumeration or exponential space is necessary.

4 A Cost-Preserving Primitive Abstraction

With $a_i = c_i = w_i$, the retry game matches the optimum over all causal primitive programs.

LEMMA 4 (ONE FAST PASS). *For $K \subseteq \mathcal{A}(T)$,*

$$V(T, b) \leq \max_{\substack{F \subseteq K \\ |F| \leq b}} \left\{ \sum_{i \in F} w_i + V((T \setminus K) \cup F, b - |F|) \right\}. \quad (4)$$

PROOF. Attempt each member of K fast once, deferring failures, then continue optimally. Availability persists. Each $F \subseteq K, |F| \leq b$, is a possible failure set with prefix cost $\sum_{i \in F} w_i$ and residual $((T \setminus K) \cup F, b - |F|)$. This feasible policy's worst cost bounds the optimum. $\square$

Define the active set and potential of a primitive state by

$$T(x) = \{i : x_i \in \{U, C, L\}\}, \quad (5)$$

$$\Phi(x, b) = \sum_{i: x_i \in \{U, L\}} w_i + V(T(x), b). \quad (6)$$

Let D be actual completed jobs and $Q = \{i : x_i = LC\}$. Every $q \in Q$ was ready at preparation, so $\text{Pred}(q) \subseteq D$. Thus $D \cup Q$ is an ideal and $T = J \setminus (D \cup Q)$ remains a valid upset. It is a *virtual* residual set: for $a \rightarrow b$, $a = LC$ and $b = U$, the game on $T = \{b\}$ permits b , but physical readiness still forbids its kernel. Conversely, every physically ready job in T belongs to $\mathcal{A}(T)$, since its predecessors lie in actual D . Every cached job is physically ready, and the cached members of any acquisition batch form an available antichain.

The potential provisionally omits valid guarded preparations LC , but must restore their obligations if released instead of freely committed.

THEOREM 5 (PRIMITIVE VALUE EQUALITY). *For the interface of Section 2, the minimum worst accumulated work from all unprepared objects is*

$$\text{OPT}_{\text{primitive}}(J, B) = \sum_{i \in J} w_i + V(J, B).$$

An optimal retry controller attains this value through serial macros: preparation and conditional commit, or acquisition, guarded preparation and commit.

PROOF. Fast prepares and conditionally commits, discarding after retained-preparation failure; protected acquires, prepares and commits. Both macros execute physically ready jobs, capture exact successful outputs and publish milestones with the game's costs/outcomes. Their availability at every all- U unfinished boundary gives the upper bound.

For the lower bound, we show that every primitive action admits a legal Nature outcome with immediate cost plus successor potential at least the current potential. Write $T = T(x)$ and remove unchanged baseline terms.

Kernel and cached-validation actions concern physically ready jobs, hence available members of T . Unguarded preparation exchanges its w_i baseline for equal cost. Acquisition/release of unprepared objects leaves the potential unchanged, including before readiness. Guarded preparation removes the baseline and job for $w_i + p_i$, using the protected bound $V(T, b) \leq p_i + V(T - i, b)$.

Conditional commit or acquisition/validation of C gives successor terms $V(T - i, b)$ on success and $w_i + V(T, b - 1)$ on invalidation to U or L . Their maximum bounds $V(T, b)$ by the fast option. Nature may invalidate even when retained failure is also offered. At $b = 0$, success suffices since $V(T, 0) = 0$.

Committing LC changes no potential. Releasing it restores i to T , increasing the potential by Lemma 1. Discarding LC also restores its baseline; discarding C restores only the baseline. Cached inspection and conditional uncomputed-snapshot acquisition offer a potential-preserving match branch.

Batch acquisition changes no term for selected U jobs. Its cached subset $K \subseteq \mathcal{A}(T)$ admits the one-pass policy. Failure set F adds baseline $\sum_{i \in F} w_i$ and leaves active set $(T \setminus K) \cup F$. Equation (4) supplies the witness, also for empty K or zero budget.

Atomic fresh completion uses the protected bound. Cached completion gives $V(T - i, b)$ on a match and $w_i + p_i + V(T - i, b - 1)$ on mismatch. Their maximum bounds $V(T, b)$: a feasible policy tries i fast, protects it immediately on failure, then continues optimally on $T - i$. At zero budget a match suffices.

Choose each outcome after observing the action. Telescoping bounds completion cost by the initial potential; noncompletion costs infinity. This covers arbitrary job changes and retained guards. Serial macros give equality. $\square$

Equality concerns the all-unprepared boundary. The potential is only a lower bound at intermediate states: one cached job with $w = c = 2$, $p = 1$, $b = 1$ has $\Phi = 1$ but remaining optimum 2. Nature can invalidate its first use, forcing work at least 2; conditional commit followed by an unguarded retry attains 2. Thus Φ is not an exact state quotient.

Transfer conditions. Exact CP/DP policies inherit Theorems 5 and 6. A refinement may restrict guard groups: executable serial macros at stated prices give the upper bound, while equality also needs realizable lower-bound outcomes and consistent shared-guard pricing. Nonnegative holding costs preserve the lower bound; equality needs unchanged witness costs. Positive-cost failed acquisition has a counterexample in Section 10.

5 From Failure Tokens to Actual Writes

Repeated stale rejection needs no new write, and parent writes preserve captured outputs. We bound *actual successful external writes*.

Java 17's `ConcurrentHashMap` stores persistent keys and immutable identity-equal wrappers. Writes install fresh identities without reuse, since conditional replacement accepts identity or equals equality [18, 19]. Fast execution reads, computes unprotected and conditionally replaces; protection evaluates inside compute. Java 17 specifies atomicity and exactly one callback invocation; inspected source evaluates/assigns under the bin monitor [17, 19]. Kernels terminate without exceptions, return non-null outputs, and keys are never removed.

Cached fallback prepares outside compute, then reuses a matching identity or recomputes under protection. The exact successful conditional/callback result becomes the milestone, never a later `map.get`. Calls perform no cross-key map work and hold no foreground guard between invocations. These conditions define the callback subclass; exposed-monitor programs need the general interface contract.

THEOREM 6 (A BOUND ON ACTUAL EXTERNAL WRITES). *Use persistent keys, immutable fresh identities without reuse, and Section 2's opaque-kernel/readiness/captured-output contract, with no initial preparation. A single foreground uses fresh reads, preparation, identity-conditional replacement, free inspection/discard and atomic fresh/cached callbacks. Even cached matches perform no other-key, other-foreground-kernel or direct-guard work; no guard survives a call. Every commit validates or uses a guarded result. Primitives finish, the foreground is scheduled and at most B external writes occur. Minimum worst counted work is*

$$\min_P \sup_{E: \# \text{writes}(E) \leq B} \text{Cost}(P, E) = \sum_i w_i + V(J, B),$$

where w_i is the counted cost of a whole kernel, including its captured-parent processing. Noncompletion has infinite cost. This equality holds whether the keys use distinct or shared bins.

PROOF. Fix any admissible P . Classify a key as C when it retains a current-identity result, and U otherwise; copying or retaining stale results creates no fresh capability. At callback boundaries the unfinished keys form upset $T = J \setminus D$. Set

$$\Psi(x, \beta) = \sum_{i: x_i = U} w_i + V(T, \beta), \quad \beta = B - \# \text{writes already made}.$$

The environment writes only before a selected conditional comparison/callback acquires its monitor. Selected kernels and valid cached commits are physically ready, hence available in T . Writing

changes only the selected input identity, preserving captured parents. The foreground holds no guard at this boundary, so writes finish even with colliding bins.

For a valid cached conditional commit, the environment chooses no write or one fresh-identity write. Removing common baseline terms, the successor potentials are $V(T-i, \beta)$ and $w_i + V(T, \beta-1)$. Their maximum bounds $V(T, \beta)$. For a valid cached callback, the second term becomes $w_i + p_i + V(T-i, \beta-1)$, and the cached-fallback inequality in Theorem 5 applies. At $\beta = 0$, no write suffices. Fresh preparation pays for removing its w_i baseline; fresh protected completion uses $V(T, \beta) \leq p_i + V(T-i, \beta)$. Preparing an old snapshot cannot create a current capability. A stale cached fallback is a fresh protected computation requiring no additional write. Repeated validation of that same stale result is a zero-write stutter, not a new budget charge. Free inspection does not alter this semantic classification; discarding the only current result increases the potential.

Callbacks read captured parents directly, with no other foreground work under incidental protection. Thus other-key effects end at the next boundary. Each operation admits a zero/one-write continuation preserving cost plus Ψ , chosen from observed history and current action. At most B writes are inserted; telescoping gives the lower bound.

The emitted controller rereads after every failure. A failed replacement requires an external write between read and comparison; these serial intervals are disjoint. Failures therefore inject into distinct writes, giving at most B failures even with other-key writes. The observed remaining-failure allowance may exceed β but remains valid. Serial macros attain the bound in either layout. $\square$

Multiple snapshots are permitted, without unrestricted Java or elapsed-time optimality. For independent guards, Φ from Equation (6) applies: invalidate a valid cached key before acquisition, or the entire failed batch subset from Equation (4) before its first acquisition. Stale rejection remains distinct from new writes.

Native data dependence and cost. Authored kernels fold captured predecessor seeds and every immutable-array entry with fixed integer arithmetic. A separate writer installs fresh wrappers. These generally noncommuting transformations make children depend on actual predecessor results.

To instantiate integer retry prices (c_i, p_i) , let $k_i = |\text{Pred}(i)|$ and choose array length $\ell_i = 8c_i - k_i > 0$ on the selected inputs. One kernel invocation counts $k_i + \ell_i = 8c_i$ work units, including every parent-seed fold. Set $d = \text{lcm}_i c_i$ and $q_i = dp_i/c_i$. If W is all foreground kernel work and L_i is that work for job i inside its callback, the objective is

$$dW + \sum_i q_i L_i = 8d \times \text{the corresponding abstract total cost.}$$

This uniformly scales the declared work metric, excluding timing, allocation, initialization, monitor/API instructions, background work and scheduling. Shared-bin effects do not multiply L_i by incidentally blocked keys.

6 Compiling the DAG Game for All Budgets

We specialize a reflected recurrence to concave child curves, eliminating explicit budget iteration at each completion ideal without fixing a topological order.

6.1 The Capped Recurrence

Fix a nonempty residual set S and a cheapest available fast job i . Write

$$g_b = V(S-i, b), \quad h_b = \min_{j \in \mathcal{A}(S)} \{p_j + V(S-j, b)\}, \quad c = c_i.$$

By Lemma 2, the full Bellman equation reduces to

$$v_0 = 0, \quad v_b = \min\{h_b, \max(g_b, c + v_{b-1})\}, \quad b > 0. \quad (7)$$

Here h can choose different protected jobs across budgets. Equation (7) is the jump projection in Burdzy et al. [21], Equations (2.12)–(2.13), with driving path $c[t]$ and barriers $g_{\lfloor t \rfloor}, h_{\lfloor t \rfloor}$. Their Theorem 2.6 gives the general explicit solution; Equation (2.8) gives the uncapped running maximum in Equation (9). The following slope-floor/cap formula is a short concavity corollary, proved directly below. Write $\Delta f_b = f_b - f_{b-1}$; discrete concavity means nonincreasing differences.

LEMMA 7 (CAPPED CONCAVE RECURRENCE). *Suppose $g_0 = 0$, g, h are nonnegative, nondecreasing and discrete concave, and $h_b \geq g_b$ for every b . For $c > 0$, define*

$$D_0 = 0, \quad D_b = \sum_{t=1}^b \max(c, \Delta g_t). \quad (8)$$

Then the solution of Equation (7) is $v_b = \min(h_b, D_b)$, and is nondecreasing and discrete concave. The result permits $h_0 > 0$.

PROOF. Concavity gives the identities

$$D_b = \max_{0 \leq k \leq b} \{g_k + c(b - k)\} = \max(g_b, c + D_{b-1}). \quad (9)$$

Indeed, the successive differences of $g_k - ck$ are nonincreasing. Its maximum over a prefix includes exactly its initial positive differences; adding cb gives Equation (8).

Induct on b . If $h_{b-1} \geq D_{b-1}$, the induction hypothesis and Equation (9) immediately give $v_b = \min(h_b, D_b)$. Otherwise choose k attaining the maximum for D_{b-1} . Since $h_{b-1} \geq g_{b-1}$ but $D_{b-1} > h_{b-1}$, $k < b - 1$. From $h_k \geq g_k$ we obtain

$$h_{b-1} - h_k < c(b - 1 - k).$$

The next difference of concave h is at most that earlier block's average, so $\Delta h_b < c$. Thus $h_b < c + h_{b-1}$ and Equation (7) selects h_b . The same strict block inequality holds for g , so its later slopes are below c and $D_b = D_{b-1} + c > h_b$. Hence the formula also holds in this branch. The strict branch cannot occur at $b - 1 = 0$.

The slopes of D are nonnegative and nonincreasing. Pointwise minima preserve nondecreasingness and the discrete concavity inequality: $2f_b \geq f_{b-1} + f_{b+1} \geq \min_f f_{b-1} + \min_f f_{b+1}$, then minimize the left side. $\square$

THEOREM 8 (DAG RETRY-CURVE IDENTITY). *For every residual set S , $V(S, \cdot)$ is a nonnegative nondecreasing discrete-concave sequence. With g, h, c as above,*

$$V(S, b) = \min \left\{ \min_{j \in \mathcal{A}(S)} [p_j + V(S - j, b)], \sum_{t=1}^b \max(c_i, \Delta V(S - i, t)) \right\}. \quad (10)$$

Its eventual constant value is $P(S) = \sum_{j \in S} p_j$.

PROOF. Induct on $|S|$, starting from the zero empty-set curve. By induction all shifted child curves are concave and nondecreasing, hence so is their minimum h . Subset monotonicity and each feasible protected action give $g_b \leq V(S, b) \leq h_b$, without presupposing the new formula. Lemma 7 therefore applies.

Protecting all jobs bounds the value by $P(S)$. For $b \geq \sum_{j \in S} \lceil p_j / c_j \rceil$, Nature can give each job its allocated failures before fast completion. Each then pays its premium through protection or failures, regardless of ready-job order. This proves eventual equality. $\square$

Policy extraction and ties. At runtime, compute the protected minimum from the available children. If it equals $V(S, b)$, protection of an attaining job is optimal; otherwise choose the cheapest available fast job. At $b = 0$, fast completion is free in residual cost. This is the prototype's protection-first tie policy and uses the full encoded values.

An optimal policy can stay on a fast job until success. Lemma 7's strict cap $h_{b-1} < D_{b-1}$ implies $h_b < c + h_{b-1}$, making fast suboptimal at b . Thus optimal fast at b implies $v_{b-1} = D_{b-1}$ and fast remains optimal. Repeating proves the claim. Fast-first ties give a fast prefix and protected suffix starting at $\tau = \min\{b \geq 1 : h_b < D_b\}$; its protected job may change.

6.2 Encoded Curves and the Compiler

Curve triples (s, v, r) encode $v + r(b - s)$ from integer budget s to the next breakpoint, merging equal adjacent slopes. A completed curve ends in an infinite constant tail.

The compiler enumerates root-reachable upsets and processes every child before its parent. For each nonempty S , find available jobs and their cheapest fast job i . Shift each child $V(S - j, \cdot)$ by p_j and take their minimum h . Replace the suffix of $g = V(S - i, \cdot)$ with slopes at most c_i by one slope- c_i segment, giving D . Store $\min(h, D)$, available-job identities and i .

Refine profiles at union breakpoints and floor/ceiling neighbors of affine crossings, then minimize and merge equal slopes. Both neighbors capture the one-step slope at a noninteger crossing; tails are included.

For N ideals and K segments, output has K entries and at most nN action identities. Runtime consults child curves with an unfinished-set identifier and budget; dense expansion can still have $(B + 1)N$ states.

Checking values and policies. The original checker refines at every pairwise affine crossing. A later checker instead sweeps candidate/child breakpoints and candidate breakpoints shifted by one. On each integer interval, subtract the candidate value from every action. Protected actions become affine lines; fast actions become maxima of two lines.

PROPOSITION 9 (DIRECT BELLMAN AND POLICY CHECKING). *Require every action difference to be nonnegative, and require the zero sets of all protected actions plus the stored fast action to cover every integer in the interval. Together with complete state/action coverage, valid curve shapes, zero-budget boundaries and constant-tail checks, these conditions certify all values and the protection-first stored policy.*

PROOF. A line's nonnegativity needs only endpoints. A two-line maximum additionally needs the floor/ceiling neighbors of its own crossing. Its zero set is where both lines are nonpositive and at least one is zero; each constituent contributes an integer interval or singleton. Merge these zero sets, accepting adjacent integer intervals even when their continuous interiors differ. Nonnegativity proves no action improves the candidate; coverage proves an allowed policy action attains it. If any protected action attains, the runtime chooses one; otherwise the stored fast action must attain. Bellman induction on $|S| + b$ gives the unique values and optimal policy. The last interval has only constant constituents, so one point covers its infinite tail. $\square$

For one state let a be its ready-job count, M its constituent profile entries (counting its own twice), and I the merged intervals. Heap merging and zero-set sorting cost $O(M \log(a + 2) + Ia \log(a + 1))$ arithmetic operations, plus shape validation. Materialized boundaries use $O(M + a)$ auxiliary space. The checker imports no constructor and directly checks every fast action, including stored-policy attainment; it trusts arithmetic and the game, not a mechanically verified abstraction.

THEOREM 10 (A SUPPORTING BASIS AND TIGHT RUN BOUND). For $C_S = \{0\} \cup \{c_j : j \in S\}$ and every integer $b \geq 0$,

$$V(S, b) = \min_{s \in C_S} \{\alpha_{S,s} + sb\},$$

omitting unused slopes, with integer $0 \leq \alpha_{S,s} \leq P(S)$. If d positive costs are distinct in S , its sampled profile has at most $2d$ positive runs. The $2n$ bound is tight even on chains. All emitted profiles together have at most $(n+1)2^n$ entries, including constant tails; construction/checking take $2^n \text{poly}(\lambda)$ bit time for binary input length λ .

PROOF. Suppose a child $g_b = \min_{\ell}(a_{\ell} + s_{\ell}b)$ has such a basis. Let $A = \max_{q \in \mathbb{Z}_{\geq 0}}(g_q - cq)$ and t be its first maximizer. It exists because g eventually becomes constant. Concavity gives $\Delta g_k > c$ through t and $\Delta g_k \leq c$ afterwards. Hence D_b from Equation (8) equals g_b through t and $cb + A$ thereafter, and

$$D_b = \min\{cb + A, \min_{\ell: s_{\ell} > c} (a_{\ell} + s_{\ell}b)\}.$$

For $b < t$, every line attaining g_b has slope above c , or it would imply $g_{b+1} \leq g_b + c$; also $cb + A \geq g_b$. For $b \geq t$, each retained line satisfies $a_{\ell} + s_{\ell}b \geq g_t + s_{\ell}(b - t) \geq cb + A$. This proves the identity, including $t = 0$ and an empty retained set.

The empty curve is the zero line. Protected actions shift child intercepts by their premiums; flooring retains child slopes and adds c_i with $A \in [0, P(S - i)]$. Equation (10) takes their union/minimum. Keeping the smallest intercept per slope closes induction, including zero premiums and repeated costs.

The continuous supporting envelope has at most $d + 1$ pieces and d breakpoints. Integer sampling adds at most one bridging unit interval per noninteger breakpoint, giving $2d$ positive runs. Summing $2|S| + 1$ over all subsets gives $(n + 1)2^n$. Partial protected envelopes obey the same linear bound, so existing profile operations/checks cost polynomial time per ideal. With $P = \sum p_i$, $C = \max(1, \max c_i)$, saturation occurs by $\sum_i \lceil p_i / c_i \rceil \leq P$. Starts, values and intercepts are at most P , slopes at most C ; crossing arithmetic uses $O(\log(P + 2) + \log(C + 2))$ bits, plus n -bit masks.

For tightness, take chain jobs $k = 1, \dots, n$ with $c_k = 3^k$, $p_k = 5 \cdot 3^{k-1}$. Reverse insertion preserves larger suffix slopes and appends $3^k, 2 \cdot 3^{k-1}$ once each: $2n$ strictly decreasing positive runs. $\square$

The maximum defining A is over integers: for $g_b = \min(3b, 5)$, $c = 2$, it is 1; maximizing the supporting envelope over reals gives $5/3$ and the wrong $D_2 = 17/3$ instead of 5. The sampled linear interpolant remains valid. A cover by w chains of lengths n_{ℓ} gives $N \leq \prod_{\ell} (n_{\ell} + 1)$ ideals: polynomial for fixed w (XP, not FPT in width). Two chains of lengths a, b give at most $(n + 1)(a + 1)(b + 1)$ entries by summing over prefix pairs. Arbitrary certificates additionally pay their supplied sizes. A small root profile does not make finding its coefficients easy or prove that ideal enumeration is necessary.

7 Fixed-Order Compilation and Verification

Without edges, protected insertion preserves readiness, removing the ideal dimension.

THEOREM 11 (INDEPENDENT ORDER NORMAL FORM). Fix a total order by nondecreasing c_i . For independent jobs, an optimal policy completes in this order, even compared with policies that switch jobs after failures.

PROOF. Let $G(S, b)$ be the optimum subject to the sorted completion order. Inserting a protected job j at its sorted position into an optimal policy for $S - j$ changes no other job and consumes no token. Thus $G(S, b) \leq p_j + G(S - j, b)$.

Induct on $|S|$ and then b ; the empty and zero-budget cases are zero. For the first sorted job i ,

$$G(S, b) = \min\{p_i + G(S - i, b), \max[G(S - i, b), c_i + G(S, b - 1)]\}.$$

Induction replaces all children by V . Hence $G(S, b)$ is no greater than fast i 's value, which Lemma 2 bounds by every fast action. Protected insertion and the smaller-set induction bound $G(S, b)$ by every protected action too. Bellman's minimum is therefore at least $G(S, b)$; feasibility gives the reverse inequality. $\square$

Normal costs are paid once. For $(p, c) = (4, 1)$, $(3, 2)$, $B = 2$, increasing- c order gives residual 3 versus 4 for increasing p/c .

7.1 Suffix Deficits and Slope Packing

For a suffix curve f with $f_0 = 0$, the current job induces

$$(R_{p,c}f)_0 = 0, \quad (R_{p,c}f)_b = \min\{p + f_b, \max[f_b, c + (R_{p,c}f)_{b-1}]\}. \quad (11)$$

Write $r_b = f_b - f_{b-1}$. For concave f , Theorem 8 specializes to

$$(R_{p,c}f)_b = f_b + \min\left\{p, \sum_{k=1}^b (c - r_k)_+\right\}. \quad (12)$$

Subtracting f_b from the slope-floor/cap formula gives Equation (12), for any prescribed serial order.

THEOREM 12 (PERSISTENT COMPILATION FOR A PRESCRIBED ORDER). *Fix a topological order π and integer prices $c_i > 0$, $p_i \geq 0$. Its optimal serial retry-macro excess profiles for all integer budgets and earliest protection thresholds admit a persistent certificate with $O(n \log(n + 1))$ nodes. Construction and checking take $O(n \log(n + 1))$ arithmetic operations after input validation; the checking bound concerns generated artifacts. For a unique topological order these values equal V and transfer through Theorem 5 under its premises. A prescribed order alone establishes no retained-preparation restriction theorem.*

PROOF. Starting with the zero suffix, write $s_b = f_b - f_{b-1}$. For $p > 0$, let a count initial positions with $s_b \geq c$, and set $\Delta(b) = c(b - a) - (f_b - f_a)$ for $b \geq a$. Subsequent slopes are below c , including the zero tail, so the first crossing satisfies $t > a$ and $\Delta(t - 1) < p \leq \Delta(t)$. Equation (12) retains slopes through a , uses c on $a + 1, \dots, t - 1$, places $z = s_t + p - \Delta(t - 1)$ at t , and retains later slopes. Here $s_t < z \leq c$ and support is $\max(\text{length}(f), t)$. Premium zero preserves the curve, sets $t = 0$ and reuses the identical root. Protection-first ties choose protection for $b \geq t$ and fast otherwise; failure keeps the job below its threshold.

Store canonical positive runs in immutable AVL trees with subtree lengths and areas. Deficit searches and constantly many height-sensitive splits/joins replace intervals, share untouched subtrees and coalesce boundaries. Each prepend adds at most two runs, so height and fresh-node allocation are $O(\log(n + 1))$. Keep every suffix root and serialize each reachable node once.

The verifier checks the order, prices, decreasing runs, balance, structural aggregates and backward references, with a zero terminal root. For $p = 0$ it checks equality and threshold zero. Otherwise it recomputes crossing conditions, support and four ranges: unchanged prefix, plateau, singleton and unchanged suffix. Exact comparison skips identical node IDs, compares run fragments, and otherwise expands a subtree, choosing the longer one when both fronts are subtrees. It uses no probabilistic equality test.

For generated adjacent roots let h be maximum height and K count fresh nodes. Range decomposition exposes $O(h)$ subtree/run chunks, including at most two partially clipped runs. Contract aligned common subtrees inside each unchanged range. Such a subtree is never expanded: its

counterpart is identical or inside a longer front ancestor, which expands first. Fresh/boundary structure has $O(K + h)$ nodes; canonical run equality bounds exposed old-side runs, and binary branching bounds common atoms. Each comparison costs $O(K + h) = O(\log(n + 1))$. Structural checking and suffix induction complete the proof. $\square$

Validation adds $O(|E| + n \log(n + 1))$ work. Bounds assume amortized/expected constant dictionary access and charge integer operand lengths. With $P = \sum p_i$, $C = \max(1, \max c_i)$, integers need $O(\log(P + 1) + \log(C + 1) + \log(n + 1))$ bits: lengths/thresholds are at most P , slopes at most C . Budget storage separately needs $O(\log(B + 2))$ bits. Value queries use $O(\log(n + 1))$ and cursor decisions $O(1)$ arithmetic operations.

Arbitrary certificates pay supplied size and actual chunk visits: alternating disjoint copies of one k -run profile across k zero-premium roots causes $\Theta(k^2)$ work despite $O(k \log k)$ size. Generation counters do not certify provenance.

Sorted slope packing. Insert sorted jobs in reverse order. All positive slopes except possibly the final singleton are at least the next c . Premium zero changes nothing. Otherwise raise a final singleton $r < c$ by $\min(p, c - r)$ and deduct that amount. If premium remains, write it as $qc + s$, $0 \leq s < c$, and append q slopes c and, if $s > 0$, one slope s . Merge equal boundaries. The resulting support is the positive-premium threshold; each insertion uses constantly many run operations.

The order theorem, deficit identity and packing invariant give the exact root curve and thresholds in $O(n \log n)$ arithmetic and $O(n)$ entries: sorting dominates constantly many run operations per insertion. A cursor advances on success and a budget counter decreases on failure.

7.2 Cost-Compatible DAGs and Shared Certificates

Suppose $c_u \leq c_v$ for every edge $u \rightarrow v$. Minimum- c Kahn traversal gives a nondecreasing-cost topological order: any smaller unfinished job has a ready ancestor of no greater cost. Equal-cost ties must respect dependencies. Removing the edges gives an independent lower bound; the sorted optimum attains it legally in the DAG for every B . Dispatching this class to packing costs $O(|E| + n \log n)$ arithmetic operations, with $O(n)$ policy entries in addition to the input graph. Otherwise, unique-order DAGs use persistent compilation; remaining DAGs retain the ideal compiler for unrestricted optimality.

The packed output also admits a small all-budget certificate. Index sorted positions from zero, let $P_k = \sum_{j \geq k} p_j$, and let F be the root curve. Each insertion adds premium mass only at the positive marginal sequence's right end, possibly raising its last singleton. Removing that mass restores the previous curve, so every suffix is

$$f_k(b) = \min\{F(b), P_k\}. \quad (13)$$

Instead of trusting packing, a verifier can treat these caps as candidates and check their Bellman equations directly.

For a root with r positive runs, direct cap checking costs $O(n \log(r + 1) + n + |E| + r)$ arithmetic. Check the order/prices, descending runs, $F(0) = 0$, final value P_0 and zero terminal suffix. For each $p_k > 0$, set $a = P_{k+1}$, $z = P_k$, $u = \min\{b : F(b) \geq a\}$, $v = \min\{b : F(b) \geq z\}$, taking $u = 0$ for $a = 0$, and $\delta_b = F(b) - F(b - 1)$. Verify

$$\begin{aligned} \delta_{u-1} &\geq c & (u \geq 2), \\ \min(F(u), z) &= \min(z, \max(a, c + F(u - 1))) & (u \geq 1), \\ \delta_{\max(1, u+1)} &= \delta_{v-1} = c & (\max(1, u + 1) \leq v - 1), \\ z &\leq c + F(v - 1). \end{aligned}$$

These are Bellman before u , at u , between u, v and at saturation v . Concavity extends the marginal endpoint checks across each interval; later protection gives the constant cap. Zero budget is checked separately. Require threshold v , where protection first attains; a zero-premium job has identical adjacent caps and threshold zero. Suffix induction proves all values/policies. Prefix lengths/areas answer the constant checks per job by binary search, yielding the bound. The checker imports no generator and trusts arithmetic, the game and the order theorem.

7.3 Causal Control and Inherited Bounds

For a protected set H fixed before execution, the exact worst residual value is

$$U_H(B) = \sum_{i \in H} p_i + B \max_{i \notin H} c_i, \quad U(B) = \min_{t \in \{0, c_1, \dots, c_n\}} \left\{ \sum_{c_i > t} p_i + Bt \right\}. \quad (14)$$

The empty maximum is zero. Nature spends all tokens on a largest-cost fast job; for threshold t , protect exactly $c_i > t$. Sorting gives U in $O(n \log n)$ arithmetic, also with precedence using a topological order.

The clairvoyant allocation lower bound is

$$L(B) = \max_{k_i \in \mathbb{N}, \sum_i k_i \leq B} \sum_i \min(p_i, k_i c_i). \quad (15)$$

Nature fails each job's first k_i fast attempts, forcing its premium or allocated failures, so L bounds every DAG policy. Classical concave allocation [6] sorts each job's $\lfloor p_i/c_i \rfloor$ full c_i -chunks and one remainder. Binary multiplicities give $O(n \log n)$ arithmetic/ $O(n)$ runs; this and related robust budget calculations [7] are inherited.

The bounds can differ: $(p, c) = (2, 5), (4, 6)$ gives $L(1) = 4 < V(1) = 5 < U(1) = 6$. Divisible premiums make independent packing sort only full chunks, giving $V = L$; precedence need not preserve this. Every adversarial path of a fixed deterministic controller is reproducible by its per-job failure counts. The gap from L is causal versus advance-informed control.

8 Implementation and Evaluation

We test semantic/native correspondence (**RQ 1**), adaptive-order benefits (**RQ 2**), and representation/construction costs (**RQ 3**). Two final DAG evaluations freeze inputs/checks after exploration. All inputs are authored; the artifact separates exploratory studies, development regressions and controls.

We use an Apple M 2 (24 GiB), Python 3.10.9 (3.12.14 for later methods) and Homebrew OpenJDK 17.0.19. Retained ConcurrentHashMap. java matches upstream; the binary was not independently rebuilt. Hashes, controllers, commands, traces and raw results retain every planned success, failure, timeout, invalid or unstarted unit, without retry or exclusion.

Separate implementations evaluate scalar values, curves and comparators. Python checks Java's profile/threshold actions and resulting traces.

AI use. Codex assisted with source comparison, research design, conjecture/refutation, proofs, implementation, inputs, execution, analysis and writing. Other assistants provided read-only author-side checks, not independent blind review. Outcomes come from recorded programs/native tools; no human-participant or simulated-human evaluation occurred.

8.1 RQ 1: Semantic and Native Correspondence

Fixed complete-primitive evaluation. For $n = 1, 2, 3$, enumerate every forward-edge DAG and every $(c, p) \in \{1, 2\} \times \{0, 1, 2, 3\}$ vector, with budgets zero through four. All 21,160 roots of the 4,232

priced inputs agree among the complete primitive game, scalar recurrence and serialized/reloaded hybrid. Every admissible state passes Bellman, goal-rank and potential checks; every action satisfies its potential inequality, and saved policy choices agree.

All 4,232 units succeed under ten-second/180-second limits: 2,936 packing/1,296 ideal routes; general certificates pass all-budget checks. The 584 independent inputs repeat an earlier grid; four dependent inputs appeared in preflight. Redundant-edge graphs remain distinct. Primitive enumeration covers budgets 0–4; all 24 malformed certificates are rejected.

Fixed native evaluation. A pre-model seed generates 48 DAGs: eight per size three/four/five in two strata. Costs/premiums range over 1..12/0..24; forward edges have probability 0.35 plus an enforced first-to-last edge. One stratum sorts costs; the other decreases cost on that edge. Labels are permuted; each route has 24 inputs. Budgets $\{0, 1, 2, 4\}$ yield 192 roots, 384 bin-layout/1,920 policy cells; fixed-order selection includes all 397 orders. Section 5 supplies the native mapping.

All 19,644 pre-fixed paths match outputs, actions, generations, parent seeds, work and writes in both layouts. Four parent-postwrite controls each write once to a completed parent’s key; children use captured milestones with zero failed comparisons. Write/failure counts are separate. One vector repeats development; no adverse/missing units occur.

Preparation takes 5.033 seconds, compilation/execution 1.542; finer phase times are unavailable. Earlier selected paths remain separate.

8.2 RQ 2: Adaptive Order and Strong Comparators

Minimizing reverse compositions of R_{p_i, c_i} over *all* topological orders gives $F(S, b)$. The order knows the initial budget, precedes outcomes and has optimal adaptive modes. This compares retry macros; no retained-preparation restriction theorem is established.

Minimum size and a constructive gap. Equation (3) gives $F = V$ for $B \leq 1$. With at most three jobs, equality holds for every initial B : the concave operator admits a first job protected immediately or kept fast until success. The remaining two jobs have a common optimal order for all residual budgets, fixed by precedence or the independent theorem. Attaching it gives one fixed initial order, possibly dependent on B .

Scaling the introduction’s family by $k > 0$ gives residual $\{1, 2, 3\}$ values $(5k, 8k)$ at budgets one/two, choosing FAST 2/PROTECTED 1. Fixed suffixes 123/132 give $(7k, 8k)$ and 213 gives $(5k, 10k)$. Equation (2), with job 0 and normal work $16k$, gives $B = 2$ totals $24k/26k$. The artifact retains the search, perturbation argument and selected scalar/primitive/native checks; this establishes no population or maximum-gap claim.

The lookahead comparator uses U from Equation (14): score each available protected job by $p_i + U(S - i, b)$, and each available fast job by $\max\{U(S - i, b), c_i + U(S, b - 1)\}$. The two rules differ only in fast-first versus protection-first ties, then use c_i and job identity. The exact cached family chooses between fresh protection and preparation followed by atomic reuse/recomputation:

$$C(S, b) = \min_{i \in \mathcal{A}(S)} \{p_i + C(S - i, b), \max[C(S - i, b), c_i + p_i + C(S - i, b - 1)]\},$$

with $C(S, 0) = C(\emptyset, b) = 0$. These methods adapt to observed outcomes; the cached family’s optimum is solved exactly.

Both strict fixed-order gains in Table 1 arise in one input: $c = (4, 10, 3, 8, 7)$, $p = (23, 4, 10, 7, 17)$, $E = \{1 \rightarrow 2, 4 \rightarrow 1, 4 \rightarrow 2\}$. Including normal work 32, hybrid/best-order totals are 47/48 at $B = 2$ and 61/62 at $B = 4$, also beating both lookahead rules and cached fallback. These sparse, small gains limit practical conclusions.

Table 1. Hybrid work versus causal comparators on 192 priced roots from 48 authored DAGs, without layout duplication. Counts do not estimate population frequencies.

Comparator	Smaller	Equal	Larger	Max. reduction
Best fixed order	2	190	0	2.08%
Lookahead, fast ties	18	174	0	12.50%
Lookahead, protected ties	30	162	0	13.64%
Exact cached fallback	138	54	0	76.74%

A common protection price. Two exploratory grids with $p_i = \lambda c_i$ have 93,665 successful case-price units and seven strict gaps. Simplification gives $c = (8, 2, 4, 2, 6)$, $p = 1.5c$, $E = \{0 \rightarrow 1, 1 \rightarrow 3\}$, $B = 4$: total 48 versus 49 for the best of twenty fixed orders. A selected primitive check and 746 native paths agree. Thus heterogeneous surcharges are unnecessary for a gap; prevalence and minimum size within this restriction remain unestablished.

8.3 RQ 3: Encoded Size and Construction Cost

A 22,077-unit check passes: 105,936 floor-identity cells, 128,116 curves in 4,355 existing artifacts, 64 tight chains and two controls. Supporting-basis checks cover all integer budgets but do not replace Bellman/action checking or upgrade prior timeouts.

The original scale study uses 87 inputs from 29 chain, disjoint-chain, layer, fence and independent templates, with three small/wide price families above $2^{32}/2^{40}$. Each cold construction/checking unit has five seconds and sampled 1 GiB RSS. All 174 units remain: construction 78 successes/nine timeouts; checking 74 successes/four timeouts/nine unstarted; no failures/invalids. Wide ideal sets and large-controller checks time out, including a retained 6,561-ideal/83,042-segment output. Twenty-two checked roots have $L < V < U$ above budget 10^{10} ; this is not an optimized all-budget comparison.

Structural specialization. Compatible scaling covers 48 new DAGs up to 32,768 jobs and nine old inputs with changed code paths. All 57 compile under ten-second/1 GiB caps. Initial checking has 49 successes/eight timeouts; shared-root checking passes all saved outputs without recompilation. Maximum output is 5,356,484 bytes/30,248 runs; maximum construction/serialization and checking times are 0.415/0.365 seconds. Original receipts remain.

Persistent prescribed orders. Exploration checks 24,029 inputs, including 128 prescribed branching orders. All 44,838 shared-checker units have expected outcomes, including 20,758 adversarial certificates with valid metadata (74 accepted/20,684 rejected). On 60 authored chains, five-second/1 GiB cold construction, serialization, reload, checking and queries give 42 successes/18 timeouts with direct Bellman checking versus 36/24 for the existing hybrid. Later shared checking gives 54/6; other statuses are zero. All 60 persistent artifact hashes are unchanged. Its six timeouts are 32,768-job increasing/local-inversion inputs; compatible packing completes every increasing input. Cross-run timing is not randomized, and route-selected saved results are not a measured dispatcher run.

Specified-budget alternatives. At $B = 1$, Equation (3) gives an exact CP model: protection occupies a p_i -length interval; fast execution is a point on the premium clock. Precedence uses interval ends; fast points impose $t_i + c_i \leq K$, with total premium at most K . During certificate extraction, a point inside an interval moves before it.

Table 2. Specified-budget comparisons: success/timeout counts. All other statuses and shared-value disagreements are zero. CP/DP return a policy for the requested budget; the hybrid also constructs and verifies all-budget curves.

Method	$B = 1$ (78 roots)	$B = 2$ (48 roots)
CP-SAT	78/0	48/0
Screened DP	68/10	48/0
All-budget hybrid + check	53/25	46/2

At $B = 2$, let q_f be protected premium before a first failure at f on a root order, leaving upset R_f including f . Attach a $B = 1$ certificate of value C_f for each root fast job. The tree costs

$$\max \left\{ \sum_{i \in H} p_i, \max_{f \notin H} (q_f + c_f + C_f) \right\}.$$

Every tree is a legal policy; every adaptive policy has one no-failure order and such first-failure continuations. Thus minimizing is exact. Full permutations, order/protection conjunctions and active-remaining flags yield $O(n^3)$ CP size, not polynomial solve time. OR-Tools 9.15.6755 [23] runs one worker with a four-second solve cap; the $B = 2$ model also uses L/U and fixed-mode hints.

A screened DP combines L with optimal modes for cheapest-available/ID-first orders. Equal bounds finish immediately; child lower bounds and Lemma 2 otherwise prune actions. Separate scanners evaluate emitted fixed-order/branching policies; optimality also trusts the formulation/solver.

Fixed construction comparisons. The $B = 1$ set has 60 new roots: chains, four disjoint chains, fences, layers and independent sets at sizes 8/16/32/64 with three price families, plus 18 deliberately generated CLIQUE-reduction roots. The $B = 2$ set has 24 compositions of the four-job family (two to five blocks, three coupling patterns, ordinary/wide premiums) and 24 random-price roots with fences, layers or forward edges at sizes 4/8/12/16. Inputs and code are fixed before each run. Every unit has a five-second cold-process/1 GiB cap including import, construction, output and checking. These are authored method probes, not application frequencies.

The initial $B = 1$ DP has 56 successes/22 timeouts. Its 78-unit screened extension preserves all 234 original units, closes 62 roots without expansion and times out on ten hardness-derived roots. At $B = 2$, screening closes 21 roots; maximum expansion/output is 701 states/17 policy nodes. All 48 cold runs beat CP (2.841 versus 31.190 seconds including startup), subject to 50 ms polling resolution. Small specified budgets can avoid all-budget construction/verification.

Direct queries at larger budgets. Equation (9) gives a value oracle: binary-search the last non-negative difference of $g_k - ck$ on $0..B$ through memoized child queries. Equal L/U bounds and compatible residuals avoid search. Certificates retain protected-child values and the maximizing point/neighbor slopes, checked via Theorem 8 for requested values only.

A fixed comparison uses 48 new inputs: four DAG shapes, sizes 4/8/16/32, and small, wide-premium or wide-both prices. Each requests 1, 2, 8, $\max(1, Q/4)$, $\max(1, Q/2)$, $\max(1, Q)$, with integer floors, $Q = \sum_i [p_i/c_i]$ and duplicates retained. Under five-second cold-process/1 GiB caps, all-budget construction succeeds on 45 and times out on three; the oracle succeeds on 32 and times out on sixteen. Other statuses are zero; all 192 joint values agree. Twelve oracle timeouts use wide premiums. Joint-success cold totals 5.217/5.613 seconds support no oracle advantage. The oracle omits full policies; its online action cost is unmeasured.

Checking the emitted curves. Both checkers complete 36 saved larger certificates under five-second/1 GiB caps: old/new checking totals 14.586/5.933 seconds, cold 16.588/8.090. The new version directly checks policy attainment. This probe reruns no constructor or original timeout.

A subsequent full-pipeline run on all 48 direct-query inputs retains the caps and yields the same 45 successes/three timeouts, with 45 controller hashes and 270 values unchanged. Old/new total cold time is 35.689/26.891 seconds; common-success reload/check/query time is 14.659/5.788. All three 32-job fences time out before artifact writing. This descriptive cross-run comparison improves checking cost without expanding completion coverage; original outcomes remain.

Retained independent comparisons. Earlier independent PRISM/AND-OR studies [11] retain 130 units: 84 successes, 37 timeouts, five memory failures, four invalids. Bounds/divisibility-screened DP completes 140/144 roots versus packing 144/144 under six-second caps; all 48 ratio-one roots use the inherited shortcut. These do not compare general DAGs.

9 Related Work

Quantitative and fault-tolerant synthesis. Černý et al. [12] synthesize synchronization, including optimistic retries and guards/caches. Their Theorem 6 matches distributions/weights to preserve limit-average values; our potential gives initial total-cost equality and intermediate lower bounds. Their erratum supersedes the original hardness claim. Dumitrescu et al. [1] minimize worst fault cost in weighted reachability, expressing finite instances here. Resilience and cost/resilience synthesis are established [3, 4].

Černý et al. [13], Sections 6.1–6.2, also optimize protected-statement counts and use profiled protected-block costs in a contention/lock-acquisition performance model. Our premiums are declared, without runtime calibration. Finite shared failures, persistent completion and encoded causal retry/order policies require comparison beyond the established concern for protected computation.

Concurrent-program reduction. Gangamreddypalli et al. [14], Theorem 1, preserve original failures and reach each original successful final state or replace it by failure (P2). Here snapshot read/conditional commit do not commute unconditionally with same-key writes, and LC release restores a virtual job by inequality. A cost-preserving static/dynamic reduction remains unestablished.

Exact robust dynamic programming. Mannor et al. [2], Section 4/Theorem 5, supply Equation (2) on unfinished upsets with negated costs and horizon $n + B$: fast completes freely or spends one token and costs c_i on a self-loop; protection completes for p_i . Only uncertainty-segment endpoints matter; token-charged nominal outcomes are dominated. State/budget expansion remains explicit.

Postek et al. [24] provide exact finite-candidate DP, state expansion, causal budgeted routing and appendix reformulations (Propositions 1–2, Remarks 1–2, Section 5.1). Their code enumerates budgets/reuses saturation; our concavity specialization and primitive abstraction need separate justification.

Parametric robust optimization and allocation. Bertsimas and Sim [25], Theorem 3/Equations (15)–(19), solve additive-deviation robust binary optimization through $n + 1$ nominal problems. Their $Z(\Gamma) = \min_{\ell} \{\Gamma d_{\ell} + F_{\ell}\}$ has budget-independent F_{ℓ} and only $d + 1$ distinct-deviation cases: an inherited input-slope envelope under combinatorial constraints. Repeated failures and causal protection/order require a preserving fixed-binary-set reduction; none is established. A DAG or concavity alone is no distinction.

Classical separable-concave allocation [6, 7] supplies the inexpensive bound L , exact on the divisible independent family. Decision-dependent information and controllable uncertainty are established broader settings [9, 10].

Scheduling and hardening. At $B = 1$, coupled processing/tail modes $(p_i, 0)$ or $(0, c_i)$ minimize $\max_i (C_i + \text{tail}_i)$; precedence constrains machine completion, leaving tails overlapping. Fixed modes fall under Lawler’s [22] sequencing result. Our hardness concerns mode choices, without priority over every scheduling variant.

Izosimov et al. [15] activate offline DAG schedules from bounded-failure histories, with non-preemption, signalling and transparency. An and Yang [5] synthesize stochastic hardening under probabilistic (m, k) requirements/fixed priorities. We omit these constraints. Chrobak et al. [8], Lemma 4/Theorem 5, concentrate NUM_k faults on a target-dependent critical job, giving prescribed-order latest completion times in $O(n)$ independent of k . Their release/deadline model mandates retries; we select premium-priced protection and emit all-budget policies/certificates for counted work, without release/deadline constraints.

Bold and Goerigk [26], Section 3, fix conflict precedences, observe all durations, then choose start times using $\Gamma + 1$ DAG layers/a compact MIP. Activities complete despite delay; our failures retain an unfinished state before another causal decision. Transfer needs these semantics matched.

Rent-or-buy ordering. Multi-commodity thresholds [16] and general rental/submodular purchase costs [20] establish heterogeneous ordering; Li et al.’s Lemma 3 uses purchase/rental ratios. The direct offline map $f(H) = \sum_{i \in H} p_i$, $g(H) = \sum_{i \in H} c_i$, $T = B$ gives 6 for $(p, c) = (2, 5), (4, 6)$, $B = 1$, versus our 5. Other reductions remain unexcluded.

10 Limitations and Threats to Validity

Contract sensitivity. Optimality requires fixed opaque kernels, immutable parents, persistent completion and own-key invalidation. It excludes partial/parallel evaluation, external discharge, replenished budgets, structural changes and unmodeled exceptions.

Restricted outcomes retain the upper guarantee but can break optimality. With $a_1 = a_2 = 1$, $(p_1, c_1) = (10, 1)$, $(p_2, c_2) = (10, 10)$, $B = 1$, let job 2 fail only after job 1 completes. Increasing- c completion then costs 12, versus 3 in reverse order. A free guaranteed operation certifying that every future writer has finished instead reduces the optimum to baseline work. Finite writer counts do not exclude this certificate; a thread count or rate is not a total-write bound.

A refuted broader acquisition model. Charged API work and positive-cost failed acquisition let retained guards improve five of 384 earlier inputs, including unrestricted 242 versus macro 243. These counterexamples remain for that contract. Nonnegative holding costs preserve the lower bound; equality needs unchanged serial-witness costs.

Meaning of the evaluation. Native runs cover recorded paths; curve checking trusts the game. Finite checks provide no mechanical proof or independent review. Strongest-comparator gains are sparse; CP/DP sometimes avoid timeouts, while the value oracle fares worse. Applicability and stronger theory/comparator coverage remain open.

11 Conclusion

Primitive equality and an actual-write adversary justify encoded optimal retries, with compact fixed-order certificates, parameterized tractability, restricted hardness and a minimum adaptive-order separation.

Data availability. The portable artifact preserves inputs, controllers, outcomes/traces and refutations/corrections. Anonymous release, remaining history curation, redistribution and final author approvals remain pending.

References

- [1] Emil Dumitrescu, Alain Girault, Hervé Marchand, and Éric Rutten. 2007. *Optimal discrete controller synthesis for the modeling of fault-tolerant distributed systems*. INRIA Research Report RR-6137. https://www.researchgate.net/publication/250079523_Optimal_discrete_controller_synthesis_for_modeling_fault-tolerant_distributed_systems.
- [2] Shie Mannor, Ofir Mebel, and Huan Xu. 2012. Lightning Does Not Strike Twice: Robust MDPs with Coupled Uncertainty. In *ICML*. <https://icml.cc/2012/papers/215.pdf>.
- [3] Daniel Neider, Alexander Weinert, and Martin Zimmermann. 2020. Synthesizing optimally resilient controllers. *Acta Informatica* 57, 195–221. <https://doi.org/10.1007/s00236-019-00345-7>.
- [4] Alexander Weinert. 2019. *Optimality and Resilience in Parity Games*. PhD thesis, Saarland University. Section 6.2. https://finkbeiner.groups.cispa.de/_assets/paper.C89PMq4G.pdf.
- [5] Hyung-Chan An and Hoesok Yang. 2021. Fully Adaptive Stochastic Handling of Soft-Errors in Real-Time Systems. *IEEE Access* 9, 155058–155071. <https://doi.org/10.1109/ACCESS.2021.3128397>.
- [6] Awi Federgruen and Henri Groenevelt. 1986. The Greedy Procedure for Resource Allocation Problems: Necessary and Sufficient Conditions for Optimality. *Operations Research*. https://business.columbia.edu/sites/default/files-efs/pubfiles/4071/federgruen_greedy_procedure.pdf.
- [7] Dimitris Bertsimas and Melvyn Sim. 2004. The Price of Robustness. *Operations Research* 52, 1, 35–53. <https://www.mit.edu/~dbertsim/papers/Robust%20Optimization/The%20price%20of%20Robustness.pdf>.
- [8] Marek Chrobak, Mathilde Hurand, and Jiří Sgall. 2009. Algorithms for Testing Fault-Tolerance of Sequenced Jobs. *Journal of Scheduling*. Author version. <https://iuuk.mff.cuni.cz/~sgall/ps/faults.pdf>.
- [9] Phebe Vayanos, Angelos Georgioudis, and Han Yu. 2026. Robust Optimization with Decision-Dependent Information Discovery. *Management Science* 72, 2, 1509–1528. <https://doi.org/10.1287/mnsc.2021.00160>.
- [10] Eva Ley, Rebecca Marx, Maximilian Merkert, Tim Niemann, and Sebastian Stiller. 2026. Decision-dependent robust optimization with elementwise controllable uncertainty sets. *OR Spectrum*. <https://doi.org/10.1007/s00291-026-00856-z>.
- [11] PRISM-games developers. 2026. PRISM-games. Official tool documentation. Version 3.2.4 used in the experiments. <https://www.prismmodelchecker.org/games/>.
- [12] Pavol Černý, Krishnendu Chatterjee, Thomas A. Henzinger, Arjun Radhakrishna, and Rohit Singh. 2011. Quantitative Synthesis for Concurrent Programs. In *CAV*, 243–259. https://doi.org/10.1007/978-3-642-22110-1_20. Author publication page and complexity erratum: <https://www.microsoft.com/en-us/research/publication/quantitative-synthesis-for-concurrent-programs/>.
- [13] Pavol Černý, Edmund M. Clarke, Thomas A. Henzinger, Arjun Radhakrishna, Leonid Ryzhyk, Roopsha Samanta, and Thorsten Tarrach. 2015. Optimizing Solution Quality in Synchronization Synthesis. *arXiv preprint arXiv:1511.07163v1*. <https://arxiv.org/abs/1511.07163>.
- [14] Namratha Gangamreddypalli, Constantin Enea, and Shaz Qadeer. 2026. Reduction for Structured Concurrent Programs. *arXiv preprint arXiv:2601.13341*. <https://arxiv.org/abs/2601.13341>.
- [15] Viacheslav Izosimov, Paul Pop, Petru Eles, and Zebo Peng. 2012. Scheduling and Optimization of Fault-Tolerant Embedded Systems with Transparency/Performance Trade-Offs. *ACM Transactions on Embedded Computing Systems* 11, 3, Article 61, 35 pages. <https://doi.org/10.1145/2345770.2345773>.
- [16] Bingham Wu, Wei Bao, Dong Yuan, and Bing Zhou. 2022. Competitive Analysis for Multi-Commodity Ski-Rental Problem. In *IJCAI*, 4672–4678. <https://doi.org/10.24963/ijcai.2022/648>.
- [17] Oracle. 2021. ConcurrentHashMap, Java SE 17 API Specification. Accessed September 7, 2026. <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/ConcurrentHashMap.html>.
- [18] Oracle. 2021. ConcurrentMap, Java SE 17 API Specification. Accessed September 7, 2026. <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/ConcurrentMap.html>.
- [19] OpenJDK. 2026. ConcurrentHashMap.java, jdk 17 u source, commit 982d53c227943a511e8d61694feced764b71a6c1. Accessed September 7, 2026. <https://github.com/openjdk/jdk17u/blob/982d53c227943a511e8d61694feced764b71a6c1/src/java.base/share/classes/java/util/concurrent/ConcurrentHashMap.java>.
- [20] Ziwei Li, Bo Sun, Zhiqiu Zhang, Mohammad Hajiesmaili, Bingham Wu, Lin F. Yang, and Yang Gao. 2025. Combinatorial Ski Rental Problem: Robust and Learning-Augmented Algorithms. In *NeurIPS*. https://papers.nips.cc/paper_files/paper/2025/hash/ae3e173398d6e43fea63bcc16fbaa98-Abstract-Conference.html.
- [21] Krzysztof Burdzy, Weining Kang, and Kavita Ramanan. 2009. The Skorokhod problem in a time-dependent interval. *Stochastic Processes and their Applications* 119, 2, 428–452. <https://doi.org/10.1016/j.spa.2008.03.001>.
- [22] Eugene L. Lawler. 1973. Optimal Sequencing of a Single Machine Subject to Precedence Constraints. *Management Science* 19, 5, 544–546. <https://doi.org/10.1287/mnsc.19.5.544>.
- [23] Laurent Perron, Frédéric Didier, and Steven Gay. 2023. The CP-SAT-LP Solver (Invited Talk). In *CP, LIPIcs* 280, 3:1–3:2. <https://doi.org/10.4230/LIPIcs.CP.2023.3>.
- [24] Krzysztof Postek, Ward Romeijnnders, and Wolfram Wiesemann. 2025. Technical Note: Multi-Stage Robust Mixed-Integer Programming. *Operations Research* 73, 6, 3345–3358. <https://doi.org/10.1287/opre.2023.0520>. Online appendix;

author code.

[25] Dimitris Bertsimas and Melvyn Sim. 2003. Robust discrete optimization and network flows. *Mathematical Programming* 98, 49–71. <https://doi.org/10.1007/s10107-003-0396-4>.

[26] Matthew Bold and Marc Goerigk. 2021. A compact reformulation of the two-stage robust resource-constrained project scheduling problem. *Computers & Operations Research*. <https://doi.org/10.1016/j.cor.2021.105232>. *Accepted manuscript*.

[27] Richard M. Karp. 1972. Reducibility Among Combinatorial Problems. In *Complexity of Computer Computations*, 85–103. https://doi.org/10.1007/978-1-4684-2001-2_9. Primary paper.